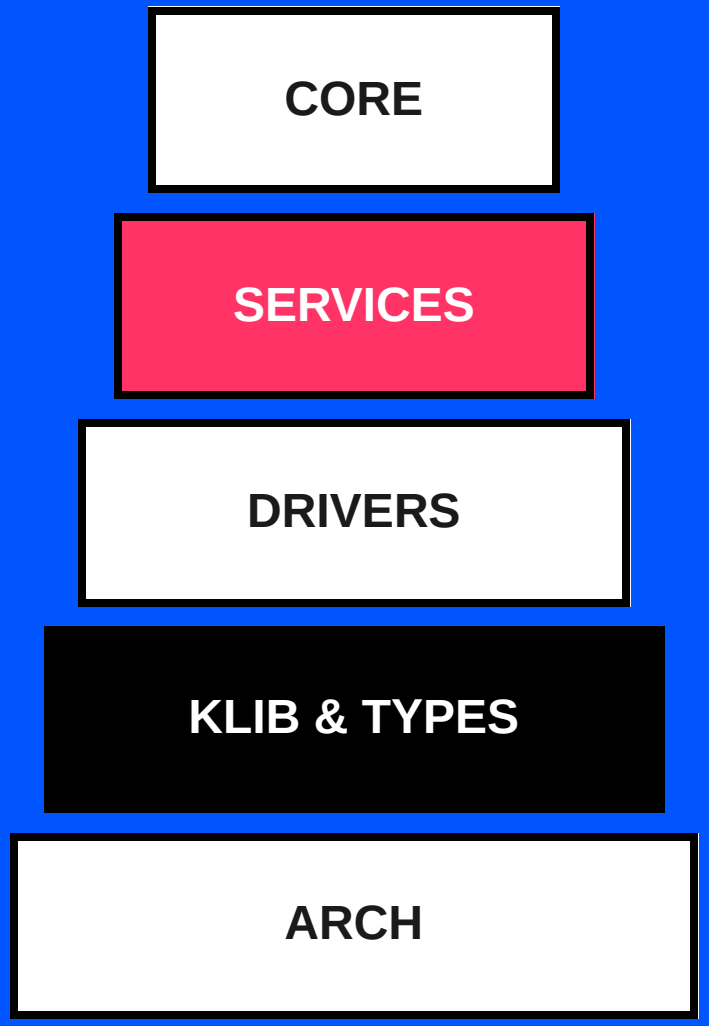
KFS-1 KERNEL ARCHITECTURE

> The current architecture is broken — here is the fix and how to enforce it.



I386 | RUST | MONOLITHIC KERNEL | LAYERED ARCHITECTURE

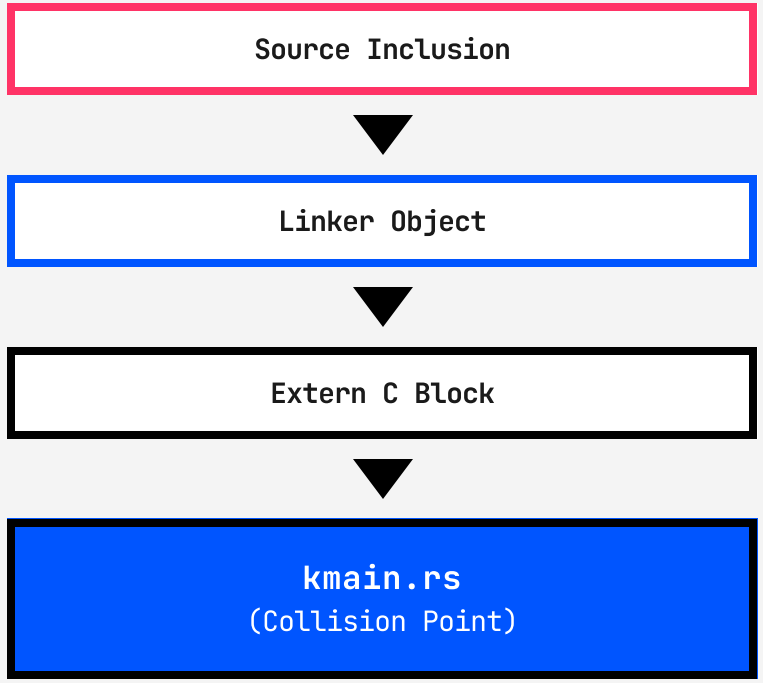
THE REPO HAS NO SINGLE ARCHITECTURE — IT RUNS THREE INCOMPATIBLE BOUNDARY MODELS AT ONCE

The KFS-1 kernel currently mixes three distinct boundary styles with no governing rule:

1. SOURCE INCLUSION (MOD / USE)	
WHERE IT APPEARS types.rs, *_impl.rs	WHAT IT IMPLIES One coherent Rust module tree

2. PER-FILE LINKED COMPONENTS	
WHERE IT APPEARS Makefile glob → separate .o per src/kernel/*.rs	WHAT IT IMPLIES Pseudo-component model

3. C ABI SYMBOL EXPORTS	
WHERE IT APPEARS vga_init, kfs_strlen, kfs_memcpy	WHAT IT IMPLIES Binary ABI between Rust files



KEY INSIGHT

These three styles encode three different answers to the same question: **"how do kernel parts relate to each other?"**

Running all three at once means there is no architecture — only accumulated habit.

KMAIN OWNS EVERYTHING, BOUNDARIES ARE ACCIDENTAL, AND EVERY FILE INVENTS ITS OWN CONTRACT

CURRENT FILE TREE & DEPENDENCIES

```
src/
  arch/i386/          ← true ABI edge (correct)
    boot.asm
    linker.ld
  kernel/
    kmain.rs          ← orchestration + subsystem (wrong)
    string.rs         ← extern "C" ABI surface (accidental)
    memory.rs         ← extern "C" ABI surface (accidental)
    vga.rs            ← extern "C" ABI surface (accidental)
    types.rs          ← excluded from Makefile glob
    string/
      string_impl.rs ← private leaf (source-included)
    memory/
      memory_impl.rs ← private leaf (source-included)
    kmain/
      logic_impl.rs  ← private leaf (source-included)
```

THE PROBLEM IN ONE BLOCK

```rust
// kmain.rs — everything mixed in one extern "C" surface
unsafe extern "C" {
    static kernel_start: u8;        // ✓ true ABI: linker symbol
    static kfs_test_mode: u8;       // ✓ true ABI: asm symbol

    fn vga_init();                  // ✗ fake ABI: Rust-to-Rust
    fn kfs_strlen(ptr: *const u8);  // ✗ fake ABI: Rust-to-Rust
}
```

No single answer to:

- What is a kernel module?
- How does one Rust subsystem call another?
- Where does a new file belong?

FOUR ARCHITECTURAL QUESTIONS THE REPO CANNOT ANSWER CONSISTENTLY

1. MODULE IDENTITY

QUESTION

What is a kernel module?

CURRENT ANSWER

> Sometimes a source file, sometimes a linked object, sometimes an ABI symbol group.

→ Impact: No module definition, no placement rule.

2. INTERNAL CALL CONTRACT

QUESTION

How should one Rust subsystem call another?

CURRENT ANSWER

> Sometimes use, sometimes linker symbol, sometimes extern "C".

→ Impact: No consistent internal call contract.

3. ABI SURFACE CLARITY

QUESTION

Which ABI edges are truly external?

CURRENT ANSWER

> Mixed in one extern "C" block alongside real linker symbols.

→ Impact: Accidental ABI hardening of internal Rust code.

4. FILE PLACEMENT

QUESTION

Where does a new feature file belong?

CURRENT ANSWER

> Wherever the Makefile glob picks it up + export by habit.

→ Impact: Mechanical placement, not architectural placement.

ONE ARCHITECTURE, ONE RULE: LAYERED RUST MODULES WITH ABI ONLY AT DESIGNATED SURFACES

THE LAYER STACK

CORE	Boot sequencing, kmain, orchestration
SERVICES	Console API, kernel-facing surface
DRIVERS	VGA text, hardware behavior
KLIB	strlen, memcpy, freestanding helpers
TYPES	KernelRange, Port, shared semantics
MACHINE	Port(u16), typed low-level primitives
ARCH	boot.asm, linker.ld, _start

↓ Dependencies point strictly downward ↓

STRATIFIED ABI RULE

REQUIRED	ARCH	_start → kmain handoff
DESIGNATED	KLIB	kfs_strlen, kfs_memcpy
ENTRY ONLY	CORE	kmain entry point
FORBIDDEN	TYPES & SERVICES	No ABI exports allowed
FORBIDDEN	PRIVATE LEAVES	Driver internals, leaf impl files

THE LAYER MODEL MUST BE VISIBLE IN THE FILESYSTEM — DIRECTORIES ARE THE ARCHITECTURE

BEFORE (CURRENT — BROKEN)

```
src/kernel/
  kmain.rs          ← peer file, owns everything
  string.rs         ← peer file, ABI surface
  memory.rs         ← peer file, ABI surface
  vga.rs            ← peer file, ABI surface
  types.rs          ← excluded by Makefile special case
  string/
    string_impl.rs
  memory/
    memory_impl.rs
  kmain/
    logic_impl.rs
```

AFTER (TARGET — CORRECT)

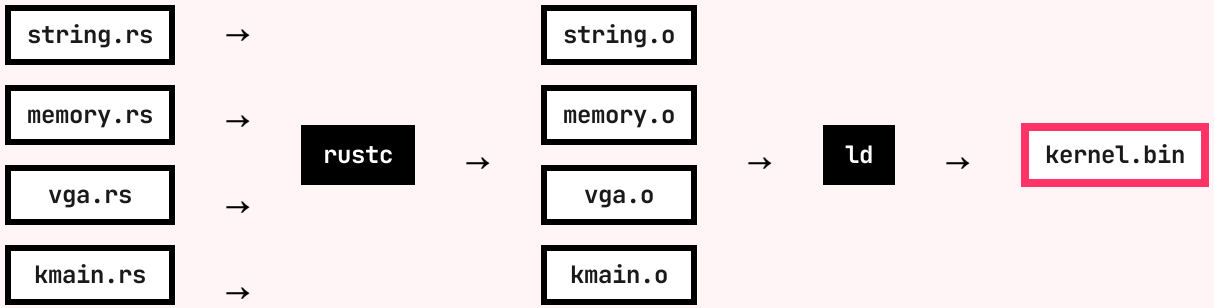
```
src/
  kernel.rs         ← single Rust crate root
  kernel/
    core/           entry.rs · init.rs · panic.rs
    machine/        port.rs
    types/          range.rs · screen.rs
    klib/           string/ (mod.rs + imp.rs)
                    memory/ (mod.rs + imp.rs)
    drivers/        vga_text/ (mod.rs + writer.rs)
    services/       console.rs
  arch/i386/        boot.asm · linker.ld · grub.cfg
```

HARD RULE

Only **core**, **machine**, **types**, **klib**, **drivers**, **services** are allowed as first-level directories under **src/kernel/**. Adding any other is an architecture violation.

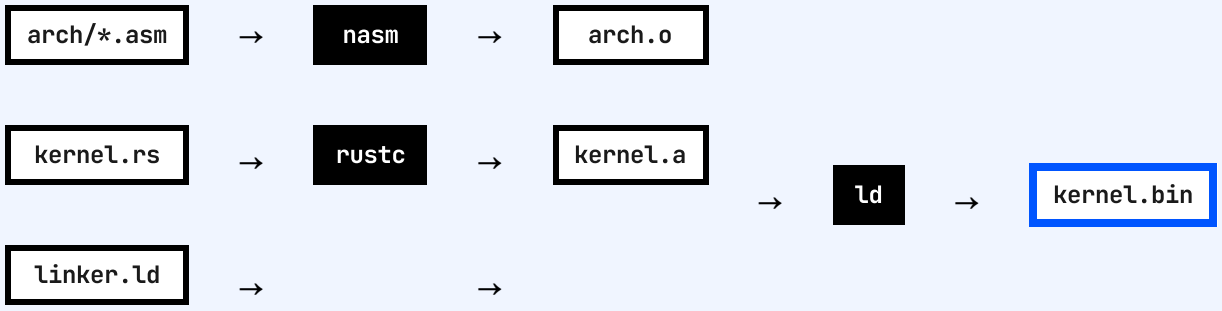
THE MAKEFILE CURRENTLY TURNS FILE PLACEMENT INTO ARCHITECTURE — ONE RUSTC CALL FIXES THAT

BROKEN BUILD (CURRENT)



Problem: File placement = component boundary. Adding a .rs file silently creates a new pseudo-component.

CORRECT BUILD (TARGET)

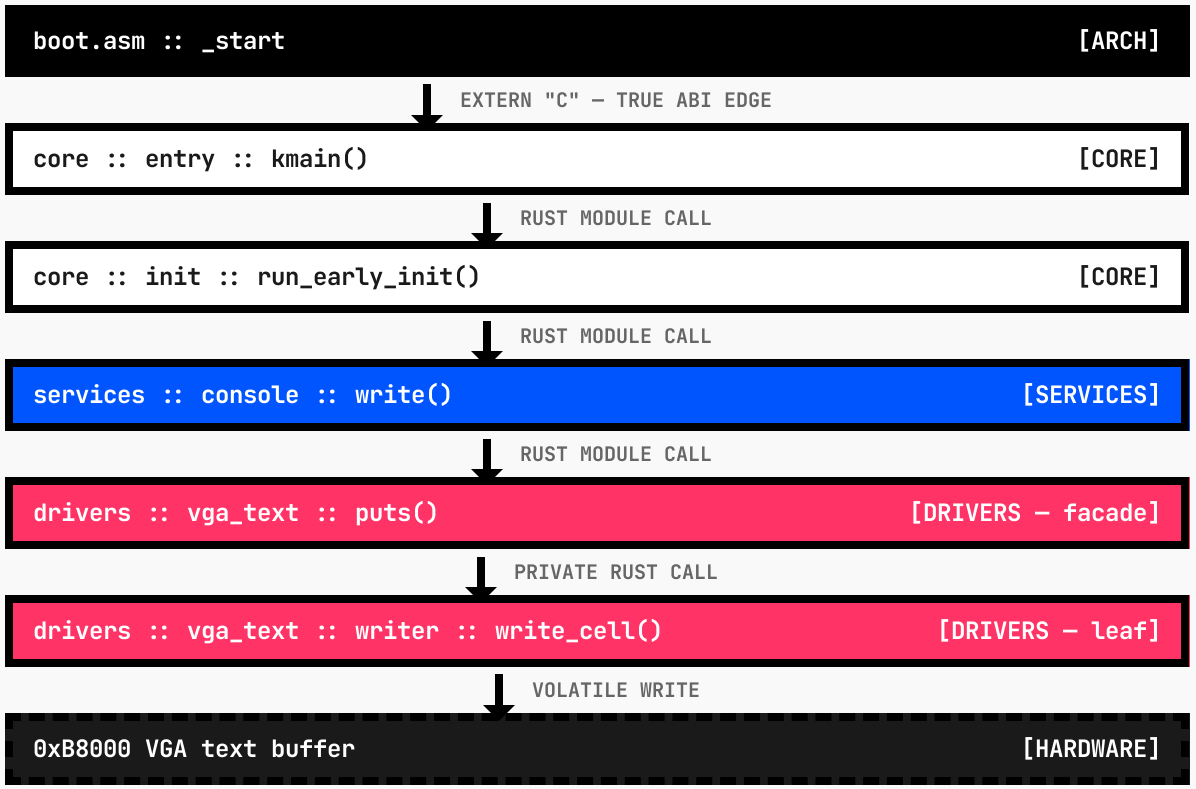


Rule: rustc is invoked exactly once for the kernel, with src/kernel.rs as the single entry point. All src/kernel/**/*.rs files are Rust modules, not build units.

```
# WRONG — per-file pseudo-component model
rust_source_files := $(wildcard src/kernel/*.rs)
```

```
# CORRECT — one Rust kernel tree
rustc --crate-type staticlib --edition 2021 \
  --emit=obj -o build/kernel.o src/kernel.rs
```

KMAIN MUST NOT TOUCH VGA MEMORY — THE RUNTIME PATH ENFORCES LAYER OWNERSHIP



✗ FORBIDDEN — core writing VGA directly

```rust
pub extern "C" fn kmain() {
    unsafe {
        (0xB8000 as *mut u16).write(0x0248);
    }
}
```

✓ CORRECT — core calls service, service calls driver

```rust
pub extern "C" fn kmain() -> ! {
    match run_early_init() {
        Ok(()) => {
            console::write(b"42\0");
            halt_forever()
        }
        Err(e) => runtime_fail(e),
    }
}
```

ONE RULE IS NOT ENOUGH — ARCHITECTURE MUST BE ENFORCED AT FOUR INDEPENDENT LEVELS

LEVEL 1: AGENT / WRITING-TIME

MECHANISM
kfs-kernel-architecture-guard skill

- Catches wrong file placement, forbidden exports, cross-layer leaks before they are written.
- Hard-fails on new peer files under src/kernel/ or #[no_mangle] outside allowlist.

LEVEL 2: HUMAN / REVIEW-TIME

MECHANISM
.githooks, CI gate, branch protection

- Catches architectural drift that "looks reasonable" locally.

Key Insight: The gate is the enforcement mechanism. The reviewer is only the last human backstop.

LEVEL 3: STATIC / BUILD-TIME

MECHANISM
scripts/tests/unit/kernel-architecture.sh

- **A. Tree shape:** only allowed first-level dirs exist.
- **B. Import/dependency:** no cross-layer leaf imports.
- **C. Symbol allowlist:** nm -g output must match exactly.
- **D. Build shape:** no per-file rustc calls.

LEVEL 4: PROOF / TEST-TIME

MECHANISM
Unit, boot, rejection, and stability tests

- **Rejection test pattern:** mutate temp tree into forbidden state → run gate → require failure.

Example: adding #[no_mangle] in imp.rs must cause gate to fail.

ELEVEN INVARIANTS THAT MUST STAY TRUE — ANY VIOLATION IS AN ARCHITECTURE REGRESSION

1. One statically linked monolithic kernel image.
2. One Rust module tree rooted at src/kernel.rs.
3. Exactly six first-level domains: core, machine, types, klib, drivers, services.
4. **arch** owns bootstrap and linker-facing concerns, not kernel policy.
5. **core** owns boot sequencing, not hardware implementation.
6. **services** own kernel-facing APIs, not hardware access.
7. **drivers** own hardware behavior, not boot policy.
8. **klib** owns freestanding reusable helpers, not device logic.
9. **types** own shared semantic meaning, not I/O.
10. Exported symbols are limited to _start, kmain, linker boundaries, and explicit low-level helper ABI.
11. Data crossing exported ABI boundaries must use explicitly stable low-level representations.

ENFORCEMENT TARGET	LEVEL 1 (AGENT)	LEVEL 2 (HUMAN)	LEVEL 3 (STATIC)	LEVEL 4 (PROOF)
FILE PLACEMENT	Agent refuses	Hook/CI blocks	Tree check	Rejection test
BUILD SHAPE	Agent refuses	Hook/CI blocks	Makefile check	Stability test
LAYER CONTRACTS	Agent labels	Hook/CI blocks	Import grep	Boot test
ABI SURFACE	Agent refuses	Hook/CI blocks	nm allowlist	Rejection test